NRES 746, Week 10, Lecture 2: Comparing Pond Models with LOO (take-home)

Answer key. Answers are in magenta sans-serif type; longer answers have a bar in the margin.

Due Monday. This uses the 60-pond survey from Lab 8 and the Stan setup from last week's take-home. Fitting all four models takes a few minutes.

Using AI: each coding question is labeled with a recommendation. After each one, circle how you actually used AI.

What to turn in: your filled-in answers and a screenshot of your `loo_compare()` output (upload in WebCampus). You don't need to submit your code.

Part 1: The model

Here is Lab 8's logistic-binomial model in Stan. `X` holds whichever standardized predictors you pass in, and `prior_sd` sets the width of the Normal(0, `prior_sd`) prior on every slope. To compute LOO, Stan must also save the **log-likelihood of each pond** at every draw, in `generated quantities`. One line is missing.

```
library(cmdstanr)
ponds <- read.csv(
  "https://kevintshoemaker.github.io/NRES-746/data/BdRegionalSurvey.csv")
X_all <- scale(as.matrix(ponds[, 2:9]))   # temp, elev, canopy, cond, area,
                                          # fish, bullfrogs, road

pond_code <- "
data {
  int<lower=1> N;
  int<lower=1> K;
  matrix[N, K] X;
  array[N] int<lower=0> n_frogs;
  array[N] int<lower=0> n_pos;
  real<lower=0> prior_sd;
}
parameters {
  real b0;
  vector[K] beta;
}
model {
  b0 ~ normal(0, 2.5);
  beta ~ normal(0, prior_sd);
  n_pos ~ binomial_logit(n_frogs, b0 + X * beta);
}
generated quantities {
  vector[N] log_lik;
  for (i in 1:N)
    log_lik[i] = ____________________________________ ;
}
"
pond_model <- cmdstan_model(write_stan_file(pond_code))

# fits the model with the predictor columns in `cols` and prior SD `s`
fit_ponds <- function(cols, s) {
  pond_model$sample(
    data = list(N = nrow(ponds), K = length(cols), X = X_all[, cols, drop = FALSE],
```

```
                n_frogs = ponds$frogs_swabbed, n_pos = ponds$bd_positive,
                prior_sd = s),
    chains = 4, parallel_chains = 4, seed = 746, refresh = 0)
}
fit_b <- fit_ponds(c(1, 7, 3), 2.5)   # Model B: temp, bullfrogs, canopy
loo_b <- fit_b$loo()
loo_b
```

1a. *(Recommended: no AI.)* Fill in the missing line. (Hint: the `model` block's likelihood, for pond `i` only. Stan's `_lpmf` functions use a bar: `f_lpmf(y | parameters)`.) Then run the code. **Check:** Model B's `looic` should be close to 221.8.

```
log_lik[i] = binomial_logit_lpmf(n_pos[i] | n_frogs[i], b0 + X[i] * beta);
```

AI use (circle one): no AI / AI helped (e.g., debugging) / AI wrote the code

1b. Why does LOO need the log-likelihood of each pond separately, rather than one total for all 60?

LOO scores how well the model predicts each pond when that pond is left out, so it needs each pond's own contribution; a single total can't be split back into ponds. (It also gives a Pareto k for each pond.)

Part 2: Four models

2a. *(AI optional.)* Fit the four models below with `fit_ponds()` and `$loo()`, and fill in the table. Model D has the same predictors as C but a much narrower prior.

Model	Predictors	prior_sd	elpd_loo	p_loo	looic	max Pareto k
A	temp	2.5	*−140.1*	*3.3*	*280.2*	*0.39*
B	temp, bullfrogs, canopy	2.5	*−110.9*	*4.3*	*221.8*	*0.28*
C	all eight	2.5	*−114.9*	*9.7*	*229.7*	*0.49*
D	all eight	0.3	*−113.7*	*8.2*	*227.5*	*0.59*

AI use (circle one): no AI / AI helped (e.g., debugging) / AI wrote the code

2b. *(AI optional.)* Run `loo::loo_compare(list(A = loo_a, B = loo_b, C = loo_c, D = loo_d))` and record each model's `elpd_diff` and `se_diff` (the best model shows 0 and 0).

Model	elpd_diff	se_diff
A	*−29.2*	*7.7*
B	*0.0*	*0.0*
C	*−3.9*	*2.5*
D	*−2.8*	*2.7*

AI use (circle one): no AI / AI helped (e.g., debugging) / AI wrote the code

Part 3: Interpret

3a. Is the best model clearly better than the runner-up? Use `elpd_diff` and `se_diff`.

> No. Model B is best, but runner-up D is behind by only 2.8 with an SE of 2.7, about one SE: within noise. Model A, by contrast, is 29.2 behind (SE 7.7), clearly worse.

3b. Models C and D have the same eight predictors. Why is D's `p_loo` (the effective number of parameters) smaller, and why might it predict new ponds better?

> D's narrow prior holds every slope near 0 unless the data push it away, so the slopes can't move freely and the model behaves as if it had fewer than its nine parameters (`p_loo` 8.2 vs. 9.7). Less freedom means less fitting of noise, so predictions for new ponds improve (here elpd -113.7 vs. -114.9).

3c. Compare the LOO ranking with the AIC table from Lab 8 Act 1. What does LOO tell you that AIC didn't?

> Same story: temperature + bullfrogs + canopy is best, all eight predictors do a little worse, and temperature alone is far behind. Model B's LOOIC (221.8) is close to its AIC (221.16). LOO adds an SE for each difference, a per-pond check of the approximation (Pareto k, all at or below 0.59 here, well under 0.7), and it works for Model D, where a prior makes "number of parameters" unclear for AIC.

Reflection

Where did you use AI, if anywhere, and what did you have to check or fix? If you didn't use it, what was hardest to get working?

Open-ended.